Instruction Format and Opcode Specification
IA-32 (32-bit x86) Two-Pass Assembler

Shruti Painapalle

Contents

1 Introduction and Scope **2**

2 IA-32 Instruction Format **2**

3 Overall Instruction Byte Diagram **2**

4 Primary 1-Byte Opcode **3**

5 `0x0F` Two-Byte Opcode **3**

6 `d` (Direction) Bit **4**

7 `s` (Sign-Extension) Bit **4**

8 Opcode Lookup Tables **4**
8.1 32-bit Register Encoding 5
8.2 Register-Encoded Opcode Table 5
8.3 Main Opcode Lookup Table 5

9 How Opcode Selection Happens **6**

1 Introduction and Scope

This document explains the instruction format and opcode-selection part of a two-pass assembler targeting the 32-bit x86 (IA-32) architecture. Instruction prefixes are outside the scope of this specification.

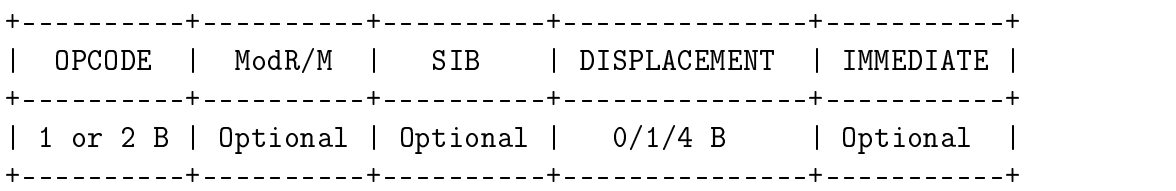
The main purpose of this section is to explain how the assembler maps an assembly instruction to the correct opcode. The opcode is selected by considering the mnemonic, operand types, operand size, direction, and the requirements of the instruction encoding.

IA-32 instructions are variable-length instructions. After excluding prefixes, an instruction can contain one or two opcode bytes followed by optional fields such as ModR/M, SIB, displacement, and immediate data.

2 IA-32 Instruction Format

An IA-32 instruction does not always have the same number of bytes. Depending on the instruction, some fields may be present while others may be absent.

The general instruction layout used in this specification is:

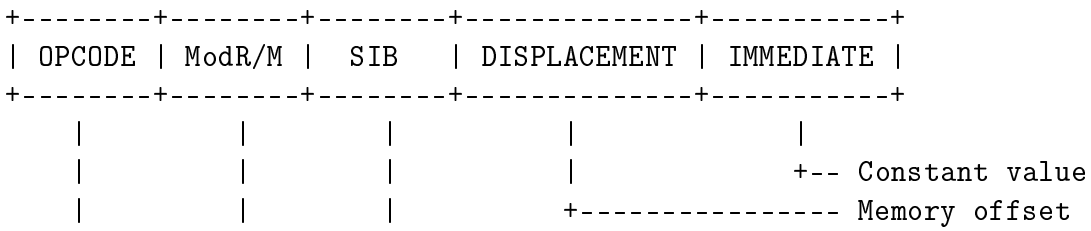


Field	Size	Description
Opcode	1 or 2 bytes	Identifies the operation.
ModR/M	1 byte	Specifies register or memory operands.
SIB	1 byte	Specifies scaled-index addressing when required.
Displacement	0, 1, or 4 bytes	Provides a constant memory-address offset.
Immediate	0, 1, 2, or 4 bytes	Stores a constant operand value.

Not every instruction contains all of these fields. The opcode and the instruction's operand form determine which additional fields are needed.

3 Overall Instruction Byte Diagram

After excluding instruction prefixes, the fields can be viewed in the following order:



```
|         |        +--------------------------- Scaled index
|         +------------------------------------ Register/memory
+---------------------------------------------- Operation
```

The fields after the opcode are optional. For example, a register-to- register instruction may require a ModR/M byte, while an instruction with a register encoded directly in the opcode may not require one.

4 Primary 1-Byte Opcode

Many IA-32 instructions use a primary one-byte opcode. A notation such as `/r` means that a ModR/M byte is required.

Instruction Form	Opcode
MOV r/m32, r32	`89 /r`
MOV r32, r/m32	`8B /r`
MOV r32, imm32	`B8+rd id`
ADD r/m32, r32	`01 /r`
ADD r32, r/m32	`03 /r`
SUB r/m32, r32	`29 /r`
SUB r32, r/m32	`2B /r`
CMP r/m32, r32	`39 /r`
CMP r32, r/m32	`3B /r`
JMP rel8	`EB cb`
JMP rel32	`E9 cd`
JNE rel8	`75 cb`
CALL rel32	`E8 cd`
RET	`C3`
NOP	`90`

Here, `/r` means that the instruction requires a ModR/M byte. For register-encoded instructions such as `MOV r32, imm32`, the register code is included directly in the opcode.

5 `0x0F` Two-Byte Opcode

Some IA-32 instructions use `0x0F` as the first opcode byte, followed by a second opcode byte.

For example, the near 32-bit form of JNE uses:

```
0F 85 cd cd cd cd
```

The general structure is:

```
+------+-------------+-------------------+
|  0F  | Second Byte | Optional Operands |
+------+-------------+-------------------+
```

For `JNE rel32`:

```
0F 85 cd cd cd cd
```

The first two bytes identify the opcode, while the following bytes contain the relative displacement.

6 `d` (Direction) Bit

Some IA-32 opcode formats contain a direction bit called `d`. It determines the direction of data transfer between the REG field and the r/m field of the ModR/M byte.

d	Meaning
0	REG is the source and r/m is the destination.
1	REG is the destination and r/m is the source.

For example:

```
MOV [EBX], EAX  ->  89 /r
MOV EAX, [EBX]  ->  8B /r
```

The two instructions perform the same general operation, but the direction of the operands is different, so different opcode forms are selected.

7 `s` (Sign-Extension) Bit

Some immediate arithmetic encodings contain an `s` bit. When `s=1`, a smaller immediate value is sign-extended to the required operand size.

This allows the assembler to use a shorter instruction when the constant fits in a signed 8-bit value.

s	Meaning
0	The immediate uses the normal operand size.
1	A smaller immediate is sign-extended to the operand size.

For example:

```
05 (8-bit) -> 00000005 (32-bit)
FF (8-bit) -> FFFFFFFF (32-bit)
```

8 Opcode Lookup Tables

The assembler uses an opcode lookup table to match an assembly instruction with the correct machine-code encoding.

8.1 32-bit Register Encoding

The three-bit register codes used in the opcode and ModR/M fields are:

Register	Binary	Decimal	Hex
EAX	000	0	0
ECX	001	1	1
EDX	010	2	2
EBX	011	3	3
ESP	100	4	4
EBP	101	5	5
ESI	110	6	6
EDI	111	7	7

8.2 Register-Encoded Opcode Table

For `MOV r32, imm32`, the opcode is calculated as:

```
B8 + rd
```

where `rd` is the three-bit register code.

Register	Code	Opcode
EAX	000	B8
ECX	001	B9
EDX	010	BA
EBX	011	BB
ESP	100	BC
EBP	101	BD
ESI	110	BE
EDI	111	BF

8.3 Main Opcode Lookup Table

Mnemonic	Operand Form	Opcode	ModR/M	Immediate
MOV	r/m32, r32	`89 /r`	Yes	No
MOV	r32, r/m32	`8B /r`	Yes	No
MOV	r32, imm32	`B8+rd id`	No	32-bit
ADD	r/m32, r32	`01 /r`	Yes	No
ADD	r32, r/m32	`03 /r`	Yes	No
SUB	r/m32, r32	`29 /r`	Yes	No
SUB	r32, r/m32	`2B /r`	Yes	No
CMP	r/m32, r32	`39 /r`	Yes	No
CMP	r32, r/m32	`3B /r`	Yes	No
JMP	rel8	`EB cb`	No	8-bit relative
JMP	rel32	`E9 cd`	No	32-bit relative
JNE	rel8	`75 cb`	No	8-bit relative

Mnemonic	Operand Form	Opcode	ModR/M	Immediate
JNE	rel32	`0F 85 cd`	No	32-bit relative
CALL	rel32	`E8 cd`	No	32-bit relative
RET	–	`C3`	No	No
NOP	–	`90`	No	No

Notation used in the table:

- `/r`: a ModR/M byte is required.
- `rd`: the register code is encoded in the opcode.
- `cb`: an 8-bit relative displacement.
- `cd`: a 32-bit relative displacement.
- `id`: a 32-bit immediate value.

9 How Opcode Selection Happens

The opcode cannot always be selected from the mnemonic alone. The assembler also has to look at the operands and the form of the instruction.

The selection process is:

```
Assembly Instruction
        |
        v
Identify Mnemonic
        |
        v
Identify Operands
        |
        v
Determine Operand Types and Size
        |
        v
Determine Direction / Immediate Form
        |
        v
Search Opcode Table
        |
        v
Select Matching Opcode
        |
        v
Generate Additional Fields if Required
        |
        v
Produce Machine Code
```

The assembler performs the following steps:

1. Read the assembly instruction.
2. Identify its mnemonic, such as MOV, ADD, SUB, or JMP.
3. Identify the operands and their types.
4. Determine the operand size.
5. Determine the operand direction when the opcode format uses a `d` bit.
6. Determine whether an immediate or relative displacement form is being used.
7. Search the opcode lookup table for a matching entry.
8. Select the opcode associated with that entry.
9. Generate ModR/M or other additional fields when required by the selected encoding.

For example, for:

```
MOV EAX, [EBX]
```

the assembler identifies the form as:

```
MOV r32, r/m32
```

The lookup table gives:

```
Opcode = 8B /r
```

The assembler then generates the required ModR/M byte for EAX and [EBX]. The final machine code is:

```
8B 03
```

Therefore, opcode selection is based on the complete instruction form, not just on the instruction mnemonic.